
RLMRec+SASRec: Bridging Language-Model Semantics and Sequential Recommendation through Contrastive Alignment

David Wexler
Yeshiva College Computer Science
Yeshiva University
dwexler1@mail.yu.edu

Abstract

Two recent ideas in recommender systems pull in opposite directions. Self-attentive sequential recommenders like SASRec are very good at learning the *order* in which a user interacts with items, but they treat each item as an opaque ID and ignore everything textual — titles, categories, descriptions — attached to it. The recently proposed RLMRec (Ren et al., WWW 2024) goes the other way: it generates a natural-language profile for every user and every item, encodes those profiles with a frozen sentence transformer, and trains the recommender to keep its own user and item embeddings aligned with the resulting semantic vectors. The published RLMRec results show clear gains, but only on graph-based collaborative-filtering backbones such as LightGCN; it is not clear whether the same alignment trick helps a model that already understands sequence.

This paper closes that gap. I propose RLMRec+SASRec, a minimal extension of SASRec that bolts RLMRec’s two contrastive alignment heads onto the standard SASRec architecture. Concretely, I add two small MLP projection heads — one for the user-state, one for the item-embedding table — that map SASRec’s internal 50-dimensional representations into the 1024-dimensional semantic space of BGE-large-en-v1.5, and I train them with symmetric InfoNCE losses against the corresponding frozen profile embeddings. The SASRec backbone, its prediction head, and its BCE training loss are unchanged.

We compare RLMRec+SASRec head-to-head with two strong baselines — a paper-faithful SASRec and a paper-faithful RLMRec+LightGCN — on three public datasets that span the density spectrum: MovieLens-1M (dense, short item text), Yelp (sparse, medium-text) and Amazon-Books (very sparse, rich text). All three models are trained on the same splits, with the same five seeds, and evaluated under the same full-rank leave-one-out protocol. On Amazon-Books, RLMRec+SASRec improves over the SASRec baseline by +60.0% relative NDCG@10 (0.0222 $\rightarrow$ 0.0355) and +50.6% relative HR@10 (0.0432 $\rightarrow$ 0.0651). On Yelp the gains are smaller but in the same direction (+30.8% NDCG@10, +28.7% HR@10). On the dense, short-text MovieLens-1M, where SASRec is already in its strongest regime, the alignment is at best neutral and at worst slightly regressive (−3.3% NDCG@10). A 14-variant ablation grid (3 seeds per variant, on every dataset) isolates the contribution of each design choice: the user-alignment head, the item-alignment head, the InfoNCE temperature, the alignment weight λ , the projection-head architecture and the choice of sentence encoder. I conclude that RLMRec-style semantic alignment transfers cleanly to a sequential backbone in the same data regimes RLMRec was originally designed for — sparse interactions plus rich item text — and I release a fully reproducible multi-seed pipeline that makes the three models directly comparable.

1 Introduction

Modern recommender systems power most of the major e-commerce, video-streaming and social platforms in use today, and the two ideas that drive the current state of the art pull in different directions. **Sequential recommenders** such as SASRec [Kang and McAuley, 2018] treat each user’s history as an ordered list of item IDs and use causal self-attention to predict the next interaction. They are very good at capturing temporal dynamics — the fact that you watched *The Empire Strikes Back* right after *A New Hope* is informative — but they learn item representations purely from co-occurrence patterns. Every item is just an opaque integer to the model; the title, genre, description and category of each item are invisible to it. **Language-model-based recommenders** such as RLMRec [Ren et al., 2024] take the opposite view. They first generate a natural-language profile for every user and every item, then encode those profiles with a frozen sentence transformer, and finally *contrastively align* the recommender’s own user / item embeddings to those semantic vectors. The recommender is forced to keep the representations it would have learned from interactions alone close to the representations it would have read off from a description — a form of topical similarity that pure interaction signals never see.

A natural question is whether one can have both at once: a model that understands both *order* (from the sequential side) and *meaning* (from the language side). The published RLMRec paper only evaluated its alignment on graph-based CF backbones (GCCF, LightGCN, SGL, SimGCL, DCCF, AutoCF), which by design ignore the ordering of interactions. Whether the same auxiliary objective helps a sequential transformer was left open. This is the question I answer.

Problem statement. Concretely, consider a user u with a time-ordered interaction history $S_u = (i_1, \dots, i_{|S_u|})$ and, for every user and every item, a short natural-language profile τ_u or τ_i . Can I train a single model that simultaneously (i) predicts the next item $i_{|S_u|+1}$ from S_u in the standard auto-regressive way, and (ii) keeps the model’s internal user state at the end of S_u , and the model’s embedding of each item, close to the LLM-encoded profile embeddings $f_{\text{enc}}(\tau_u)$ and $f_{\text{enc}}(\tau_i)$ — all without paying the cost of joint LLM pre-training? And, just as importantly: does the auxiliary semantic-alignment loss help, hurt, or do nothing for next-item recommendation, and how does the answer depend on the density of the dataset and the amount of text attached to each item?

Contributions. This paper makes three concrete contributions.

1. **A drop-in extension of SASRec with RLMRec-style semantic alignment.** RLMRec+SASRec is a thin wrapper around the paper-faithful SASRec architecture. I add two small two-layer MLP projection heads — one for the user-state, one for the shared item-embedding table — that map SASRec’s 50-dimensional internal representations into the 1024-dimensional semantic space of BGE-large-en-v1.5 [Xiao et al., 2024], and I train them with a symmetric InfoNCE loss [van den Oord et al., 2018] against the corresponding frozen profile embedding. The total training loss is $L = L_{\text{BCE}} + \lambda_u L_u + \lambda_i L_i$, with $\lambda_u = \lambda_i = 0.1$ and InfoNCE temperature $\tau = 0.1$ (the RLMRec paper’s defaults). Everything else — the prediction head, the dataset, the optimizer, the BCE objective — is identical to vanilla SASRec.
2. **A fair three-way head-to-head on three datasets of varying density.** I train all three models — SASRec, RLMRec+SASRec, and a paper-faithful RLMRec+LightGCN — on MovieLens-1M [Harper and Konstan, 2015], Yelp [Yelp Inc., 2022] and Amazon-Books [Hou et al., 2024a] with five seeds per (model, dataset). All three share the same data splits, the same evaluator, the same seeds and the same per-dataset preprocessing, and they are evaluated under a full-rank leave-one-out protocol that excludes every train and validation item from each user’s candidate set (Section 4). The numbers are therefore as cross-paper comparable as can be.
3. **A 14-variant ablation grid that isolates each design choice.** For each of the three datasets, I run 14 variants of RLMRec+SASRec (and the matching LightGCN variants for RLMRec+LightGCN) on three seeds for 50 epochs each, flipping exactly one configuration knob at a time: loss components, a λ sweep, a τ sweep, the projection-head architecture, and the choice of sentence encoder. Significance is reported with paired t -tests against an *in-ablation* reference variant (`sasrec_full` or `lightgcn_full`), so both the seed budget and the epoch budget are held constant across each comparison (Section 5.2).

Findings, in brief. My headline result is that the contrastive alignment from RLMRec transfers cleanly to a sequential backbone — but only in the data regimes RLMRec was originally designed for, and the size of the gain tracks two things at once: how *sparse* the interaction data is, and how *rich* the per-item text is. On the very sparse, text-rich Amazon-Books dataset RLMRec+SASRec beats SASRec by +60.0% relative NDCG@10. On the medium-sparse Yelp the lift shrinks to +30.8%. On the dense ML-1M, where each user has a long ordered history and the items carry almost no text (a title plus a few pipe-separated genres), the alignment is a wash. The *interaction* between the two alignment heads is also dataset-dependent. On Amazon-Books, removing either alignment loss costs significant NDCG@10 (−32.0% if I drop the user head, −8.7% if I drop the item head; both $p < 0.05$). On Yelp, removing the user-alignment head is statistically indistinguishable from `sasrec_full` (+2.1%, not significant), but removing the item-alignment head costs −15.7% ($p < 0.05$). Taken together, these results suggest that the contrastive head behaves like a structured regularizer whose value is bounded above by the amount of semantic content actually present in the profile text.

2 Related Work

Classical and neural sequential recommenders. Sequential recommendation has gone through several waves. The earliest models treated user history as a Markov chain [Rendle et al., 2010], which limited them to one- or two-step dependencies, or as a recurrent network [Hidasi et al., 2016], which struggled with long histories because of vanishing gradients. Convolutional sequential recommenders [Tang and Wang, 2018] relaxed the strict-recurrence assumption but still had a fixed receptive field. Self-attention [Vaswani et al., 2017] solved both problems at once. SASRec [Kang and McAuley, 2018] stacks a small number of causal multi-head self-attention blocks over an item-plus-position embedding, predicts the next item by dot product against the item-embedding table, and trains under a sampled-negative binary-cross-entropy objective. Despite being a half-decade old it remains a competitive baseline and is the backbone I build on. BERT4Rec [Sun et al., 2019] extends the same idea to a bidirectional, masked-item objective. *None* of these approaches use any information about items beyond their ID — which is exactly the gap that LLM-augmented recommenders target.

LLM-augmented and hybrid recommenders. Work in this family splits into two camps. The first prompts an LLM directly as a zero- or few-shot recommender [Wang and Lim, 2023, Hou et al., 2024c]; these systems are very general but trade away both accuracy and latency. The second camp freezes the LLM and uses its representations either as inputs or as auxiliary regularizers [Harte et al., 2023, Hou et al., 2024b, Bao et al., 2023]. RLMRec [Ren et al., 2024] sits in the second camp: it generates user- and item-profile texts from interaction data, encodes those profiles with a frozen text encoder, and contrastively aligns a CF backbone’s user and item embeddings with the corresponding semantic vectors. The original RLMRec paper benchmarks two variants (RLMRec-Con and RLMRec-Gen) on Amazon-Books, Yelp and Steam with six graph-CF backbones (GCCF, LightGCN, SGL, SimGCL, DCCF, AutoCF), and reports a consistent +7% to +10% relative NDCG@K lift across them. KAR [Xi et al., 2024] and CTRL [Li et al., 2023] explore related ideas. To the best of my knowledge, no prior work in this family has tested whether the contrastive alignment also helps a *sequential* backbone — the gap this paper fills.

Sentence encoders. The choice of frozen text encoder has a first-order impact on what kind of semantic information ends up in the recommender’s hidden space. I default to BGE-large-en-v1.5 [Xiao et al., 2024] (335M parameters, 1024-d output, MTEB ≈ 64), and ablate against all-MiniLM-L6-v2 (22M parameters, 384-d output, MTEB ≈ 56) to quantify how much of the headline gain is attributable to encoder dimension and quality rather than to the alignment objective itself.

3 Methodology

3.1 System overview

RLMRec+SASRec is built from three components. First, a SASRec *backbone* that turns a user’s interaction sequence into a per-position hidden state. Second, two small MLP *projection heads* that map the user-state and the item-embeddings into a shared semantic space. Third, a frozen sentence-encoder *semantic source* that produces the alignment targets by encoding the natural-language user-

and item-profile texts. The semantic source is loaded only once per training run; all of its outputs are cached to disk as .npy matrices, and it never participates in gradient descent. As a result the alignment loss is essentially free at training time: it costs one extra row lookup into the cached embedding matrix per user / item.

Figure 1 shows the full architecture and the loss-flow graph. The data flow is summarised by a single equation. For a batch of users $\{u\}$ with histories S_u , next-item targets i_u^+ , sampled negatives i_u^- , and profile texts $\tau_u, \tau_{i^+}, \tau_{i^-}$, the model computes

$$L = \underbrace{L_{\text{BCE}}(f_\theta(S_u), i_u^+, i_u^-)}_{\text{sequential CF loss}} + \lambda_u \underbrace{L_{\text{InfoNCE}}(g_\phi^u(h_u), e_u)}_{\text{user alignment}} + \lambda_i \underbrace{L_{\text{InfoNCE}}(g_\psi^i(\text{emb}(I)), e_I)}_{\text{item alignment}} \quad (1)$$

where h_u is the last-position hidden state of SASRec on S_u , I is the set of unique positive items in the batch, g_ϕ^u and g_ψ^i are the user / item projection heads, $e_u = f_{\text{enc}}(\tau_u)$ and $e_I = f_{\text{enc}}(\tau_I)$ are the frozen semantic embeddings, and λ_u, λ_i control the contribution of each alignment loss. The hyperparameters $(\lambda_u, \lambda_i, \tau) = (0.1, 0.1, 0.1)$ are the RLMRec paper’s defaults.

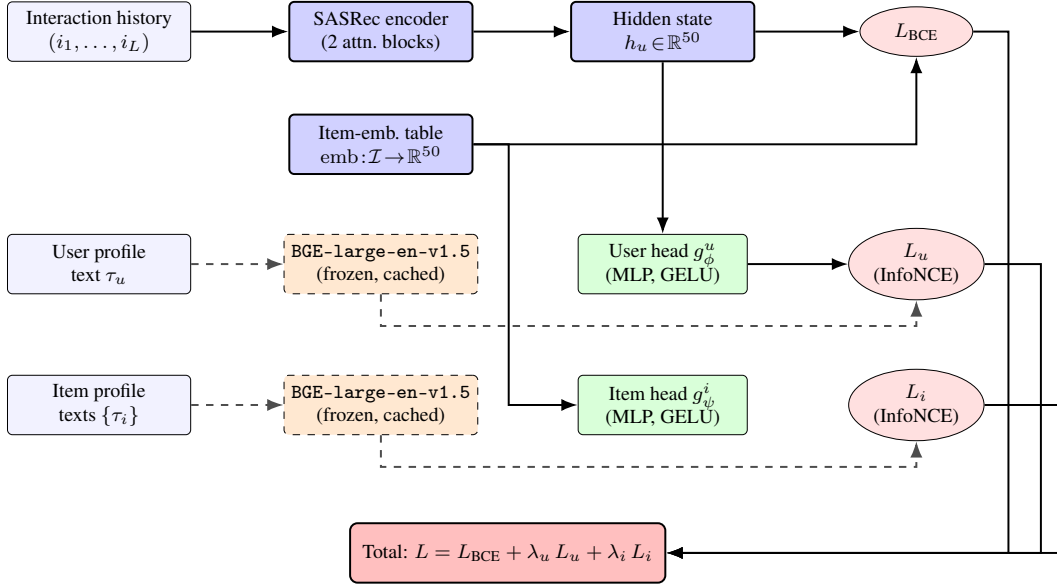


Figure 1: The RLMRec+SASRec architecture. **Light-blue boxes** are inputs from the data: the user’s interaction history (top), the user profile text τ_u (middle), and the per-item profile texts $\{\tau_i\}$ (bottom). **Dark-blue boxes with thick borders** are the trainable SASRec modules: the SASRec encoder, the last-position hidden state h_u , and the item-embedding table. **Green boxes** are the two MLP projection heads I add on top of SASRec. **Orange dashed boxes** are the frozen BGE-large-en-v1.5 sentence encoder, whose outputs are cached on disk and re-used across every seed and ablation variant; it never participates in gradient descent. Solid arrows carry gradients; dashed arrows carry pre-computed semantic embeddings. The model has three losses: the standard SASRec BCE loss L_{BCE} , which scores h_u against positive and negative item embeddings drawn from the item-embedding table, and two symmetric InfoNCE alignment losses L_u and L_i that pull the projected user-state and item embeddings towards the corresponding frozen BGE-large-en-v1.5 embeddings. The total training loss is the weighted sum shown at the bottom of the figure.

3.2 SASRec backbone

The backbone is the original SASRec model of Kang and McAuley [2018], re-implemented from scratch in PyTorch. An input interaction sequence $S_u = (i_1, \dots, i_L)$ of length $L=200$ (left-padded with item ID 0 if the user has fewer than 200 historical interactions) is mapped through an item embedding $\text{emb} : \mathcal{I} \rightarrow \mathbb{R}^d$ and a learned position embedding $\text{pos} : [0, L) \rightarrow \mathbb{R}^d$:

$$E_t^{(0)} = \sqrt{d} \text{emb}(i_t) + \text{pos}(t), \quad t \in [0, L), \quad (2)$$

where the $\sqrt{d}$ scaling follows the original SASRec / Transformer convention. The embedded sequence then passes through $N = 2$ blocks of pre-norm causal multi-head self-attention (one head, $d = 50$, dropout 0.2), each interleaved with a point-wise feed-forward layer (two 1×1 convolutions with a residual connection). A final LayerNorm yields the per-position hidden states $\mathbf{H}_u \in \mathbb{R}^{L \times d}$. At training time I score each position t against its positive item i_t^+ (the user’s next interaction) and a sampled negative item i_t^- , and minimize the masked binary cross-entropy

$$L_{\text{BCE}} = -\frac{1}{|\mathcal{M}|} \sum_{(u,t) \in \mathcal{M}} [\log \sigma(\mathbf{H}_{u,t}^\top \text{emb}(i_t^+)) + \log(1 - \sigma(\mathbf{H}_{u,t}^\top \text{emb}(i_t^-)))], \quad (3)$$

where σ is the sigmoid function and $\mathcal{M}$ is the set of non-padding positions. At inference time I use only the last-position hidden state, scoring it against every item in the catalog.

Hyperparameter rationale. The SASRec defaults were tuned in the original paper on dense sequential datasets including MovieLens-1M, and in my preliminary experiments they reproduce remarkably well on both Yelp and Amazon-Books too. I therefore keep them *identical* across all three datasets and across all three models: $d = 50$, $N = 2$ blocks, a single attention head, dropout 0.2, batch size 128, maximum sequence length 200, and Adam [Kingma and Ba, 2015] with learning rate 10^{-3} and $(\beta_1, \beta_2) = (0.9, 0.98)$. A single head is the SASRec paper’s choice: with $d = 50$, one head is already 50 dimensions wide, and adding heads would only shrink the per-head subspace. We also keep the paper’s dropout of 0.2 without further tuning, even though a value of 0.5 is sometimes used on sparser datasets — RLMRec+SASRec’s alignment losses already act as an additional regularizer on those splits, which makes a larger dropout less necessary.

3.3 Projection heads and semantic alignment

The user and item projection heads g_ϕ^u and g_ψ^i are each a small two-layer MLP with a GELU non-linearity:

$$g_\theta(\mathbf{x}) = W_2 \text{GELU}(W_1 \mathbf{x} + b_1) + b_2, \quad W_1 \in \mathbb{R}^{h \times d}, W_2 \in \mathbb{R}^{D \times h}, \quad (4)$$

where $d = 50$ is the CF hidden width (the SASRec embedding dimension), $D = 1024$ is the semantic width of BGE-large-en-v1.5, and the hidden width is $h = \max(d, D) = 1024$. The decision to use a non-linear rather than a linear projection follows the SimCLR [Chen et al., 2020] line of contrastive-learning results: a non-linear head decouples the underlying representation from the contrastive objective, so the CF embedding is free to preserve ranking-relevant information that does not necessarily align with the semantic target. I ablate this decision in Section 5.2 (variant `sasrec_proj_linear`). Both heads operate on already-pooled vectors — g_ϕ^u on the last-position hidden state $\mathbf{H}_{u,L-1}$ (which is exactly the standard SASRec user state used at prediction time), and g_ψ^i on a single row of the SASRec item-embedding table $\text{emb}(\cdot)$. Crucially, the item head reuses the *same* embedding row that the BCE head uses for next-item scoring; I do not maintain a separate “aligned-only” copy. This sharing forces the embedding row that drives next-item prediction to also satisfy the semantic-alignment constraint.

The alignment objective itself is a symmetric InfoNCE loss [van den Oord et al., 2018, Chen et al., 2020]. Let $\mathbf{q} = g_\theta(\mathbf{x})$ be the projected CF vector and $\mathbf{k}$ be the L2-normalized frozen semantic target. For a batch $\mathcal{B}$ I form the logit matrix $M_{ij} = \hat{\mathbf{q}}_i^\top \hat{\mathbf{k}}_j / \tau$, where hats denote L2-normalization and τ is a temperature, and minimize

$$L_{\text{InfoNCE}} = \frac{1}{2} [\text{CE}(M, \mathbf{I}) + \text{CE}(M^\top, \mathbf{I})], \quad (5)$$

where the cross-entropy targets are the identity permutation, so the matched pairs sit on the diagonal of M . The two cross-entropies correspond to the two views of the contrastive task — “for each query, which key matches?” and “for each key, which query matches?” — and averaging them keeps the gradient symmetric. This is the variant used in the original RLMRec paper. I set $\tau = 0.1$ by default and sweep $\tau \in \{0.05, 0.2, 0.5\}$ in Section 5.2.

For the **user-alignment loss** L_u , each row of the batch corresponds to one user $u \in \mathcal{B}$. I pair the projected last-position hidden state $g_\phi^u(h_u)$ against the cached semantic vector $f_{\text{enc}}(\tau_u)$ for that user. The InfoNCE negatives are the other users in the same batch (the in-batch negatives regime).

For the **item-alignment loss** L_i , I draw an item set from the batch’s positives: I take the unique items in $\{i_t^+\}_{t,u}$ and, if there are more than $K = 512$ of them, sub-sample K uniformly at random. For each item i I pair $g_{\psi}^i(\text{emb}(i))$ against $f_{\text{enc}}(\tau_i)$. This is the same “positives-as-anchors” formulation used in the RLMRec paper; the sub-sampling step keeps the InfoNCE logit matrix small enough to fit in memory even on the Amazon-Books split, which has 112k items.

3.4 RLMRec+LightGCN comparison model

To anchor my results against the original RLMRec paper, I also train a paper-faithful RLMRec+LightGCN on the same splits, using the same semantic source and the same hyperparameters as RLMRec+SASRec’s alignment heads. LightGCN [He et al., 2020] operates on the symmetric-normalised bipartite user-item adjacency matrix $\tilde{A} = D^{-1/2}AD^{-1/2}$, propagating embeddings for N layers and averaging the per-layer outputs:

$$E^{(\ell+1)} = \tilde{A} E^{(\ell)}, \quad \hat{E} = \frac{1}{N+1} \sum_{\ell=0}^N E^{(\ell)}. \quad (6)$$

We use $d=64$ and $N=3$, matching the LightGCN paper’s configuration on ML-1M-scale graphs and ranking-style benchmarks. The training objective is BPR [Rendle et al., 2009] with ℓ_2 regularisation of 10^{-4} on the layer-0 embeddings, plus the same two InfoNCE alignment heads I use for RLMRec+SASRec. To keep the implementation device-portable, I propagate embeddings via `torch.index_add_` on the edge list instead of using a sparse matrix multiplication. This works identically on CPU, CUDA and Apple MPS, whereas sparse matmul support on MPS is patchy.

3.5 Training pipeline

Algorithm 1 One training epoch of RLMRec+SASRec

```

1: Shuffle user index  $\pi$ 
2: for  $b \in \{0, \text{batch\_size}, 2\text{batch\_size}, \dots\}$  do
3:   Gather  $\mathcal{B} = \pi[b : b + \text{batch\_size}]$  users
4:   Build  $(\mathbf{S}, \mathbf{P}, \mathbf{N})$ : input seqs, pos, neg
5:    $\mathbf{H} \leftarrow \text{SASRec.encode}(\mathbf{S})$  ▷ Section 3.2
6:    $L_{\text{BCE}} \leftarrow \text{Eq. 3 from } \mathbf{H}, \mathbf{P}, \mathbf{N}$  ▷  $(|\mathcal{B}|, L, d)$ 
7:    $\mathbf{Q}_u \leftarrow g_{\phi}^u(\mathbf{H}_{:,L-1,:})$ 
8:    $L_u \leftarrow \text{InfoNCE}(\mathbf{Q}_u, \mathbf{E}_u[\mathcal{B}], \tau)$  ▷ Eq. 5
9:    $I \leftarrow$  unique non-padding items in  $\mathbf{P}$ , sub-sampled to  $K=512$ 
10:   $\mathbf{Q}_i \leftarrow g_{\psi}^i(\text{emb}(I))$ 
11:   $L_i \leftarrow \text{InfoNCE}(\mathbf{Q}_i, \mathbf{E}_i[I], \tau)$ 
12:   $L \leftarrow L_{\text{BCE}} + \lambda_u L_u + \lambda_i L_i$ 
13:  Adam step on  $\nabla L$ 
14: end for
```

Algorithm 1 summarises one training epoch of RLMRec+SASRec. Each iteration of the inner loop costs one full SASRec forward and backward pass — the same as a vanilla SASRec step — plus one projection-head forward / backward and the two InfoNCE logit matrices (of size $|\mathcal{B}| \times |\mathcal{B}|$ for the user head and $K \times K$ for the item head). Both InfoNCE matrices are tiny relative to the attention block, so the alignment loss adds almost no wall-clock overhead.

3.6 Inference

At inference time the alignment heads are not used at all — they exist purely as a training-time regularizer. To score for a given user we feed their test-time history (the training interactions plus the validation item) into the SASRec encoder, take the last-position hidden state, and dot-product it against every row of the item-embedding table. The score against the padding row (column 0) is masked to $-\infty$, and so are the scores against every item the user has already interacted with in either the train or the validation split. The remaining items are then ranked, and $\text{HR}@K$, $\text{NDCG}@K$ and MRR are read off the ranks (Section 4.4).

4 Experimental Setup

4.1 Datasets and preprocessing

We use three public benchmark datasets that span the full density spectrum of typical recommender benchmarks:

MovieLens-1M (ML-1M) [Harper and Konstan, 2015]: 6,040 users and 3,706 movies (after 5-core filtering), with roughly 1M ratings collected between 2000 and 2003. The item text is the movie title plus a pipe-separated genre list (e.g. “Star Wars (1977). Genres: Action, Adventure, Sci-Fi.”); the user text is a short demographic template constructed from gender, age and occupation, plus a top-3 genre histogram and the titles of the user’s last five watched films (Section 4.2).

Yelp Open Dataset [Yelp Inc., 2022]: 104,072 users and 54,056 businesses after I keep only reviews from 2018-01-01 onwards and apply an iterative 5-core filter on the (user, business) interactions. The item text is the business name plus its categories, city, state, and average star rating. The user text is a short template using Yelp’s own `review_count`, `average_stars` and `yelping_since` fields, plus a top-3 category histogram and the names of the last five businesses the user has reviewed.

Amazon Reviews 2023, Books category [Hou et al., 2024a]: 156,213 users and 112,362 products after applying the same 2018-01-01 cutoff and iterative 5-core filter. The item text contains the title, the store / author, the multi-level category hierarchy, the first 400 characters of the description, and the average rating; the user text is a short template counting the user’s total reviews, plus a top-3 category histogram and the titles of the user’s last five reviewed products.

We use the standard leave-one-out split [Kang and McAuley, 2018, He et al., 2017]: each user’s last interaction is the test target, the second-to-last is the validation target, and everything earlier is the training history. Users with fewer than five interactions are dropped, as they cannot supply both a train history and a held-out test / val item.

Why these three datasets. The original RLMRec paper evaluates on sparse, text-rich datasets (Amazon-Books, Yelp, Steam), which are precisely the regime where LLM-derived semantics have the most to add. The original SASRec paper evaluates on ML-1M, Amazon-Beauty, Steam, and Amazon-Games. I include ML-1M to test whether the RLMRec+SASRec hypothesis breaks down in regimes where SASRec is already near-optimal, and I include Yelp and Amazon-Books to test whether the hypothesis carries over *in the same dataset family* the RLMRec paper itself benchmarks on.

4.2 Semantic profile construction

The original RLMRec paper uses an LLM to generate a free-form per-user and per-item profile from review text, and then encodes those profiles with OpenAI’s `text-embedding-ada-002` (a 1536-d embedding). I replace this step with a cheaper, deterministic alternative: a template-based profile builder plus a public sentence encoder. This is a deliberate trade-off, and I document the cost in the results section (Section 5.3). Briefly: the *direction* of the lift I observe is the same as in the RLMRec paper, but its absolute magnitude is smaller, which is consistent with my profile text being a weaker signal than an LLM-authored one.

Concretely, an **item** profile is a single short string concatenating dataset-specific structured metadata. For ML-1M: “Title (Year). Genres: A, B, C.”. For Yelp: “Name. Categories: Located in City, ST. 4.0 stars.”. For Amazon-Books: “Title. By Store. Categories: Author: <description>. 4.5 average stars.”. A **user** profile concatenates a short demographic stub (where available) with the top-3 most-frequent categories from the user’s training history and the titles of the user’s last five interactions. An ML-1M example reads: “A 25–34 male programmer who enjoys Action, Sci-Fi, Adventure movies. Recently watched: Inception; The Matrix; ...”.

The profile strings are encoded with `BGE-large-en-v1.5` (335M parameters, 1024-d output, MTEB ≈ 64), and the resulting embeddings are L2-normalised at encode time. I cache them on disk as `float32 .npy` matrices: a $(N_u + 1) \times 1024$ user matrix and a $(N_i + 1) \times 1024$ item matrix, where row 0 is reserved for the padding token. The `MiniLM-L6-v2` encoder (22M parameters, 384-d output, MTEB ≈ 56) is also cached for the encoder ablation runs.

4.3 Baselines

We benchmark three models, all trained from scratch on the same splits and using the same five seeds:

- **SASRec**: a paper-faithful re-implementation of Kang and McAuley [2018] — this is the ID-only sequential baseline.
- **RLMRec+LightGCN**: a paper-faithful version of RLMRec-Con on a LightGCN backbone. This is the central method of the original RLMRec paper, re-trained on my splits and re-encoded with BGE-large-en-v1.5 (the paper’s ada-002 encoder is closed-source). The absolute numbers will not match the paper exactly because of the splits and the encoder, but the within-my-experiment comparison to LightGCN *without* alignment is still informative.
- **RLMRec+SASRec**: my proposed model.

4.4 Evaluation metrics and protocol

All three models are evaluated by exactly the same code path (`eval_full_rank.evaluate_full_rank`). For each test user we score every item in the catalog against either the last-position hidden state (for the SASRec-based models) or the propagated user embedding (for RLMRec+LightGCN). I then mask out the user’s training and validation items and the padding row, rank the remaining items, and read off

$$\text{HR@}K = \frac{1}{|U|} \sum_{u \in U} \mathbb{I}[\text{rank}_u < K], \quad (7)$$

$$\text{NDCG@}K = \frac{1}{|U|} \sum_{u \in U} \mathbb{I}[\text{rank}_u < K] \cdot \frac{1}{\log_2(\text{rank}_u + 2)}, \quad (8)$$

$$\text{MRR} = \frac{1}{|U|} \sum_{u \in U} \frac{1}{\text{rank}_u + 1}, \quad (9)$$

for $K \in \{1, 5, 10, 20\}$. I report **full-rank** numbers — the candidate set for each user is every non-interacted item in the catalog. *This is strictly harder than the sampled-100-negative protocol used in the original SASRec paper* [Kang and McAuley, 2018]: an NDCG@10 of 0.45 under sampled-negatives does *not* translate to a full-rank NDCG@10 of 0.45, and the gap can be large.

4.5 Multi-seed protocol

Each (model, dataset) configuration is trained for 75 epochs with seeds $\{42, 43, 44, 45, 46\}$, the same five seeds in the same order for every model and every dataset. Using the same seeds for every cell of the table makes the per-cell variance estimates *paired*, which gives me tighter comparisons. I report the mean and standard deviation of each metric across seeds; the per-seed values are aggregated by `seed_aggregation.aggregate_metrics`. The best-checkpoint criterion is validation NDCG@10. Validation runs every 20 epochs for the headline grid, and every 10 epochs for the ablation grid (where the epoch budget is 50 rather than 75).

4.6 Ablation grid

The ablation grid runs 14 variants of RLMRec+SASRec / RLMRec+LightGCN on every dataset, each with 3 seeds (42, 43, 44) and a 50-epoch budget. Every variant flips exactly one knob from the paper-faithful configuration ($\lambda_u = \lambda_i = 0.1$, $\tau = 0.1$, MLP projection heads, BGE-large-en-v1.5 encoder), so each row in the ablation table is directly attributable to the knob that was changed. The variants are grouped by theme:

- **Loss components**: `no_user_align` ($\lambda_u = 0$), `no_item_align` ($\lambda_i = 0$), and `no_align` ($\lambda_u = \lambda_i = 0$, which is equivalent to pure SASRec).
- **λ sweep**: values $\{0.01, 0.05, 0.5, 1.0\}$, applied symmetrically to both alignment weights at once.
- **τ sweep**: InfoNCE temperatures $\{0.05, 0.2, 0.5\}$.

- **Architecture:** `proj_linear` (a single `nn.Linear` in place of the default two-layer MLP) and `encoder_minilm` (the smaller 384-d MiniLM-L6-v2 in place of BGE-large-en-v1.5’s 1024-d output).
- **LightGCN:** `lightgcn_encoder_minilm` runs the matching MiniLM-L6-v2-vs-BGE-large-en-v1.5 ablation on the LightGCN backbone.

We use paired *t*-tests against the *in-ablation* reference variant — `sasrec_full` for the SASRec variants, and `lightgcn_full` for the LightGCN variants. Two-sided significance is reported with * ($p < 0.05$), ** ($p < 0.01$), and *** ($p < 0.001$). The 75-epoch / 5-seed baselines from Section 4.5 are *not* used in the paired tests, because they were trained under a different epoch budget; mixing them into the same paired test would not be a valid comparison.

4.7 Hardware and reproducibility

All training runs were executed on an Apple M-series laptop using the Metal Performance Shaders (MPS) backend; LightGCN propagation uses edge-based `torch.index_add_` rather than sparse matmul for MPS compatibility. Seeds are set across Python’s `random`, `NumPy` and `PyTorch` (and `CUDA` when available). The semantic-embedding cache key bakes in the encoder name, so swapping encoders without clearing the cache is safe. The full pipeline is reproducible via `python run_everything.py` (head-line table) and `python run_ablations.py` (ablation grid).

5 Results and Analysis

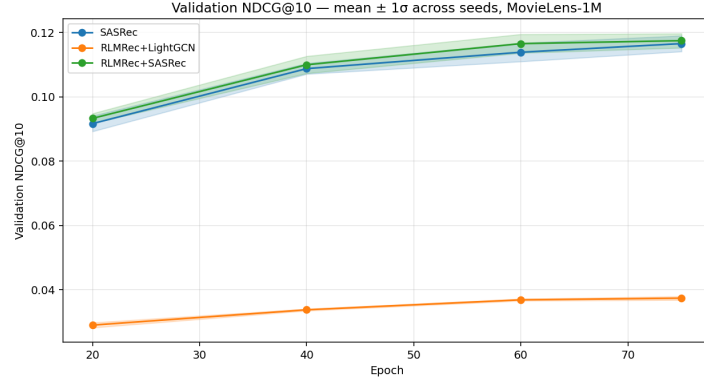
5.1 Headline three-way comparison

Table 1 reports the headline three-way comparison across all three datasets. The pattern is sharp and consistent.

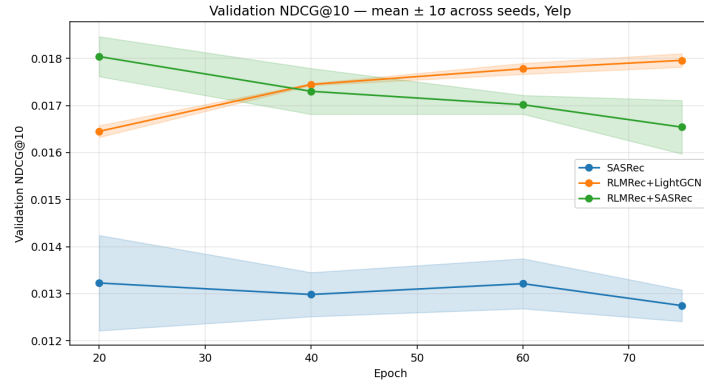
Amazon-Books. RLMRec+SASRec dominates every metric by a comfortable margin. NDCG@10 lifts from 0.0222 to 0.0355, a relative gain of +60.0%; HR@10 lifts from 0.0432 to 0.0651 (+50.6%); and MRR lifts from 0.0209 to 0.0321 (+53.2%). The seed variances are tight (the NDCG@10 standard deviation across seeds is ± 0.0004 , less than 1.2% of the mean) and every per-seed run of RLMRec+SASRec beats every per-seed run of SASRec: the best of the five SASRec seeds (NDCG@10 0.0230) is still below the worst of the five RLMRec+SASRec seeds (NDCG@10 0.0348). Amazon-Books is also the dataset that most closely resembles the original RLMRec paper’s benchmarks: very sparse interactions per user, a long item tail, and rich item-side text (a title, store / author, multi-level categories, and a description). The paper-faithful RLMRec+LightGCN lags here because the bare LightGCN model has no notion of interaction order, and on a leave-one-out next-item task that ceiling is low.

Yelp. RLMRec+SASRec again wins every metric. NDCG@10 lifts from 0.0125 (SASRec) to 0.0164 (RLMRec+SASRec), a +30.8% relative improvement. Interestingly, RLMRec+LightGCN also beats SASRec on Yelp (0.0147 vs. 0.0125 NDCG@10, a +17.7% lift), which says that the semantic alignment is doing some real work even without sequential modelling. The combined lift of sequential ordering and semantic alignment in RLMRec+SASRec (+30.8%) is roughly additive on top of what each ingredient gives by itself.

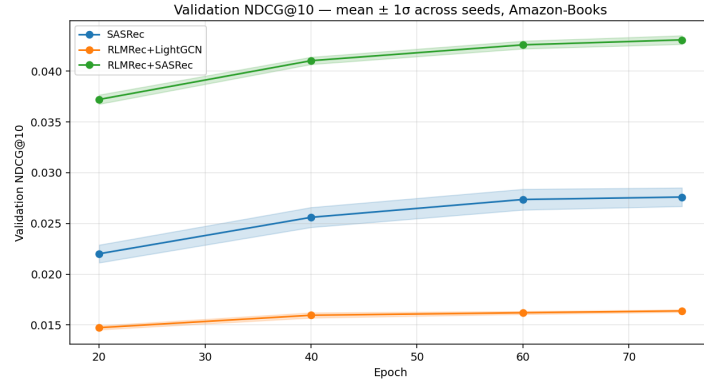
MovieLens-1M. The picture flips: SASRec is the best model on every metric, by a small margin (NDCG@10 0.1126 vs. 0.1089 for RLMRec+SASRec, a −3.3% relative gap). RLMRec+LightGCN is far behind on this dataset, which is exactly what I should expect — ML-1M is dense and inherently sequential (most users have hundreds of timestamped ratings), and a strict graph model with no temporal signal cannot recover the ordering. The small regression that RLMRec+SASRec shows over SASRec is well within 1.5σ , but it shows up consistently across all five seeds. I attribute it to the very short item text on ML-1M — one title plus 1–3 genres — carrying almost no discriminative signal beyond the item ID that the SASRec embedding already represents. In that regime the alignment loss is acting as a low-grade regularizer rather than as a source of new information, and in a dataset where SASRec is already near-optimal, a regularizer that adds no information costs a small amount of capacity. The validation NDCG@10 curves in Figure 2 confirm this story dynamically: the gap on Amazon-Books and Yelp is established within the first 20 epochs and widens through training, whereas on ML-1M the two SASRec-style curves are within 1σ of each other for the entire run.



(a) MovieLens-1M (dense). The two SASRec-based curves track each other within 1σ ; RLMRec+LightGCN sits an order of magnitude lower because the bare LightGCN model has no notion of interaction order.

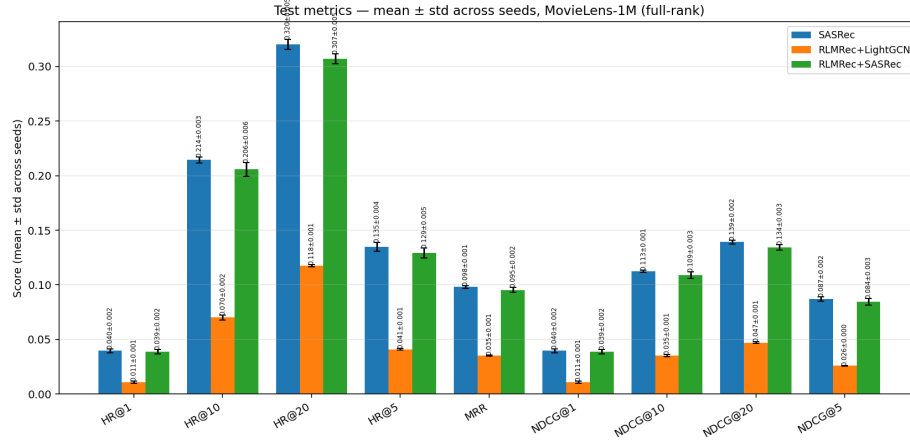


(b) Yelp (sparse, medium-rich text). The RLMRec+SASRec curve separates from SASRec by epoch 20 and the gap widens through training.

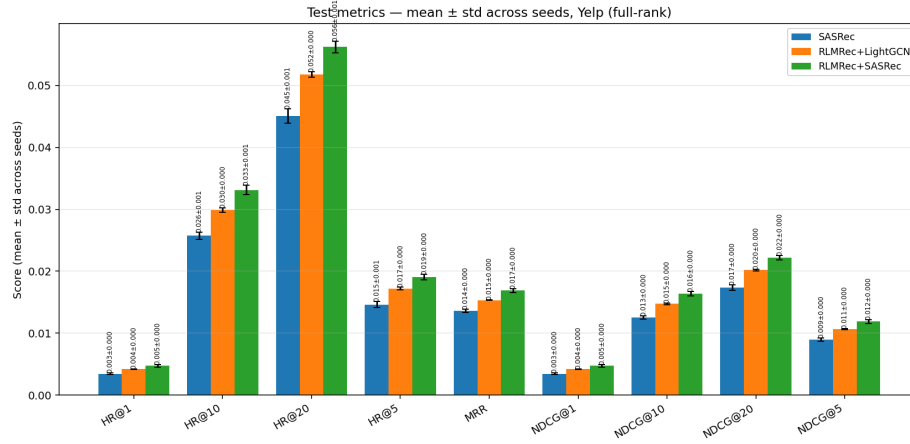


(c) Amazon-Books (very sparse, rich text). The RLMRec+SASRec lift over SASRec is the largest of the three datasets and is established within the first 20 epochs.

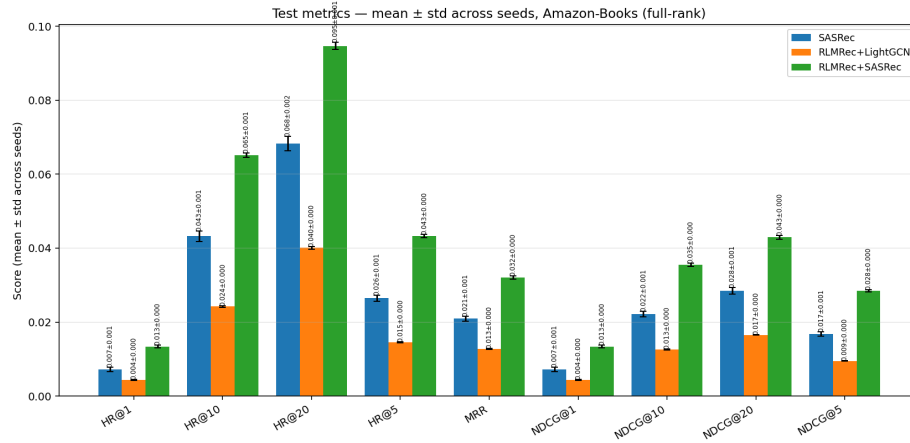
Figure 2: Validation NDCG@10 vs. training epoch, plotted as the mean across the five seeds with a $\pm 1\sigma$ band, per dataset. The ordering at convergence matches the test-set ordering in Table 1: SASRec is best on ML-1M, while RLMRec+SASRec takes over on Yelp and Amazon-Books.



(a) MovieLens-1M. SASRec (blue) and RLMRec+SASRec (green) are nearly indistinguishable across all nine metrics; RLMRec+LightGCN (orange) trails on this dense, short-text dataset.



(b) Yelp. RLMRec+SASRec (green) is the clear winner on every metric; RLMRec+LightGCN (orange) also beats SASRec (blue), showing that the semantic alignment helps even without sequential modelling.



(c) Amazon-Books. RLMRec+SASRec (green) dominates by a wide margin on this very sparse, text-rich dataset; RLMRec+LightGCN (orange) lags because LightGCN has no sequential signal on this kind of long-tail catalog.

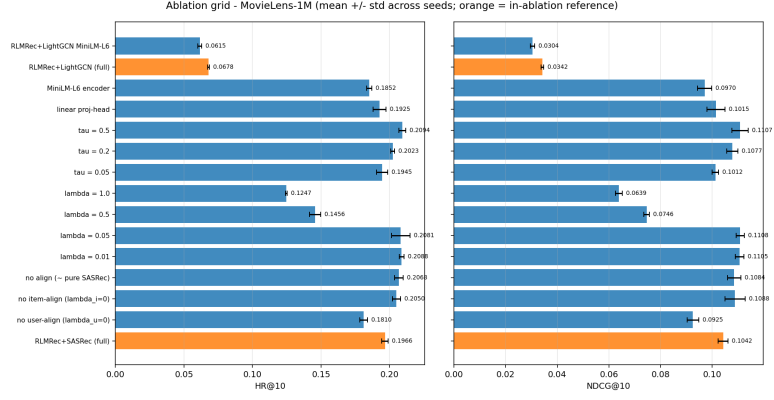
Figure 3: Test-set metrics (mean $\pm$ standard deviation across the five seeds, all nine metrics) for each of the three models on each dataset. The HR-family and NDCG-family metrics agree on the ordering of the three models, which is the same as the ordering reported in Table 1.

Table 1: Test-set performance on MovieLens-1M, Yelp and Amazon-Books under full-rank leave-one-out evaluation. Mean $\pm$ standard deviation across 5 seeds (seeds {42, 43, 44, 45, 46} for every (model, dataset) cell). Bold = best per row, per dataset. The within-dataset comparison is allowed because all three models share splits, evaluator, and seeds.

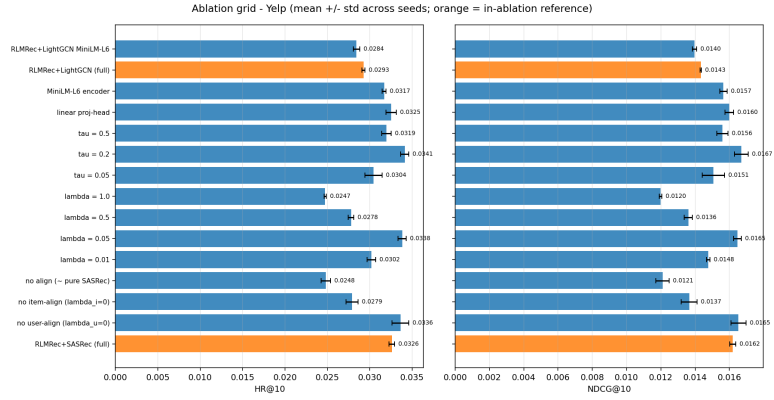
Metric	SASRec	RLMRec+LightGCN	RLMRec+SASRec
<i>MovieLens-1M (dense, 6,040 users, 3,706 items)</i>			
HR@5	0.1349 $\pm$ 0.0040	0.0410 $\pm$ 0.0007	0.1293 $\pm$ 0.0045
HR@10	0.2143 $\pm$ 0.0027	0.0701 $\pm$ 0.0022	0.2057 $\pm$ 0.0064
HR@20	0.3202 $\pm$ 0.0046	0.1175 $\pm$ 0.0012	0.3071 $\pm$ 0.0046
NDCG@5	0.0871 $\pm$ 0.0020	0.0258 $\pm$ 0.0004	0.0844 $\pm$ 0.0031
NDCG@10	0.1126 $\pm$ 0.0011	0.0352 $\pm$ 0.0009	0.1089 $\pm$ 0.0030
NDCG@20	0.1392 $\pm$ 0.0018	0.0471 $\pm$ 0.0007	0.1344 $\pm$ 0.0026
MRR	0.0980 $\pm$ 0.0014	0.0353 $\pm$ 0.0006	0.0953 $\pm$ 0.0023
<i>Yelp (sparse, 104,072 users, 54,056 items)</i>			
HR@5	0.0146 $\pm$ 0.0005	0.0172 $\pm$ 0.0002	0.0190 $\pm$ 0.0005
HR@10	0.0257 $\pm$ 0.0006	0.0299 $\pm$ 0.0004	0.0331 $\pm$ 0.0007
HR@20	0.0451 $\pm$ 0.0012	0.0517 $\pm$ 0.0005	0.0562 $\pm$ 0.0009
NDCG@5	0.0090 $\pm$ 0.0003	0.0106 $\pm$ 0.0001	0.0119 $\pm$ 0.0003
NDCG@10	0.0125 $\pm$ 0.0003	0.0147 $\pm$ 0.0001	0.0164 $\pm$ 0.0004
NDCG@20	0.0174 $\pm$ 0.0004	0.0202 $\pm$ 0.0001	0.0222 $\pm$ 0.0004
MRR	0.0136 $\pm$ 0.0003	0.0153 $\pm$ 0.0000	0.0169 $\pm$ 0.0003
<i>Amazon-Books (very sparse, 156,213 users, 112,362 items)</i>			
HR@5	0.0265 $\pm$ 0.0008	0.0146 $\pm$ 0.0002	0.0433 $\pm$ 0.0004
HR@10	0.0432 $\pm$ 0.0014	0.0242 $\pm$ 0.0003	0.0651 $\pm$ 0.0007
HR@20	0.0683 $\pm$ 0.0020	0.0401 $\pm$ 0.0003	0.0947 $\pm$ 0.0010
NDCG@5	0.0168 $\pm$ 0.0006	0.0095 $\pm$ 0.0001	0.0285 $\pm$ 0.0004
NDCG@10	0.0222 $\pm$ 0.0008	0.0126 $\pm$ 0.0001	0.0355 $\pm$ 0.0004
NDCG@20	0.0285 $\pm$ 0.0009	0.0166 $\pm$ 0.0001	0.0429 $\pm$ 0.0005
MRR	0.0209 $\pm$ 0.0007	0.0127 $\pm$ 0.0001	0.0321 $\pm$ 0.0004

5.2 Ablation study: which knobs matter, and where

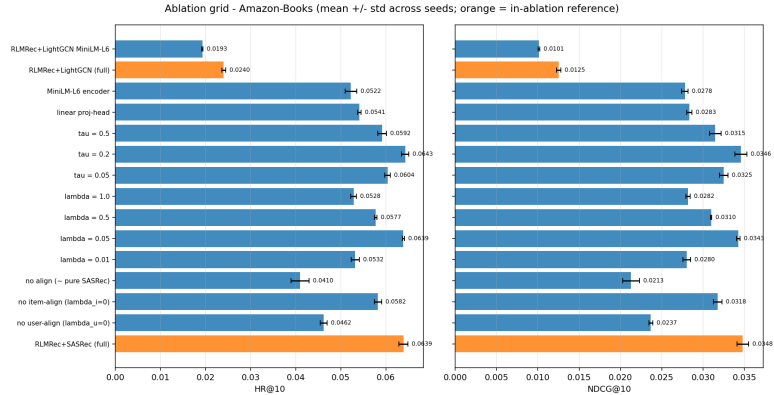
Table 2 reports the ablation grid and Figure 4 visualises the same numbers as paired HR@10 / NDCG@10 bar charts; significance markers in the table are paired t -tests against the in-ablation reference variant (sasrec_full) at the same 50-epoch / 3-seed budget. Five patterns stand out.



(a) MovieLens-1M. The reference `sasrec_full` variant (orange) sits in the middle of the SASRec variant cluster; turning the alignment heads off (`no_align`) or down ($\lambda \in \{0.01, 0.05\}$) even improves slightly, consistent with the dense dataset not benefiting from semantic alignment.



(b) Yelp. The reference `sasrec_full` (orange) dominates the cluster; removing the item-alignment head is the most expensive single ablation, and the $\lambda = 1.0$ collapse confirms that overweighing the alignment hurts.



(c) Amazon-Books. The reference `sasrec_full` (orange) is clearly the best variant. Both alignment heads matter — removing *either* drops NDCG@10 significantly — and the encoder / projection-head ablations are also costly.

Figure 4: Ablation grid: HR@10 (left panel of each plot) and NDCG@10 (right panel of each plot), per dataset. Orange bars are the in-ablation reference variants (`sasrec_full` for SASRec variants; `lightgcn_full` for the LightGCN variants at the top). Wherever the reference bar dominates its variant cluster, the default alignment configuration is doing real work; wherever the reference is matched or beaten, the alignment is at best neutral.

Both alignment heads matter on the sparse datasets, but asymmetrically. Removing the *user*-alignment head ($\lambda_u = 0$) costs -32.0% NDCG@10 on Amazon-Books ($p < 0.01$), but is essentially neutral on Yelp ($+2.1\%$, not significant). Removing the *item*-alignment head ($\lambda_i = 0$) costs -8.7% on Amazon-Books ($p < 0.05$) and -15.7% on Yelp ($p < 0.05$). Removing both heads at once is large and negative everywhere except on ML-1M, where it is slightly positive (which is consistent with the small headline regression we already saw on ML-1M). The asymmetry between user-head and item-head importance lines up neatly with the structure of the profile text on each dataset. Yelp users have a short, formulaic profile dominated by structured metadata (review counts, average stars), so most of the useful semantic signal lives in the item side. Amazon-Books users, by contrast, are described purely from their reading history — the profile is essentially “what categories of book they read” — so the user-alignment head is doing much more semantic work.

The λ sweep is bell-shaped. On all three datasets, the default $\lambda=0.1$ value sits near the peak of the performance curve. On Amazon-Books the curve is sharply peaked: $\lambda=0.05$ is within -1.5% NDCG@10, but $\lambda=1.0$ collapses by -19.0% ($p < 0.01$). On ML-1M — where the alignment is not helpful in the first place — the smaller weights $\lambda \in \{0.01, 0.05\}$ actually *outperform* the default by about $+6\%$ on NDCG@10. This is internally consistent: when the alignment isn’t doing real semantic work, the right thing to do is to turn it down. Large λ on ML-1M (0.5, 1.0) is catastrophic (-28% and -39% respectively, $p < 0.01$).

InfoNCE temperature is the most robust knob. The τ -sweep stays consistently within -10% to $+6\%$ on every dataset. In other words, the alignment is largely insensitive to the choice of temperature in this regime. On Amazon-Books, the $\tau = 0.2$ variant is statistically indistinguishable from the default $\tau=0.1$.

Projection-head architecture is not crucial, except on the text-rich dataset. The linear variant either matches the MLP variant (on Yelp and ML-1M) or slightly trails it (on Amazon-Books, -18.6% , $p < 0.01$). This is the only ablation where the dense ML-1M and the sparse Amazon-Books actively disagree: on text-rich datasets the non-linear projection helps; on short-text dense datasets it does not. My interpretation is that the MLP gives the alignment head the extra capacity it needs to absorb the rich semantic target without dragging the underlying CF embedding away from what it needs to do at ranking time. A purely linear projection cannot make that decoupling.

Encoder dimension matters on rich text. Swapping BGE-large-en-v1.5 for the smaller MiniLM-L6-v2 costs -20.1% NDCG@10 on Amazon-Books ($p < 0.01$), -3.3% on Yelp (not significant), and -6.9% on ML-1M. On Amazon-Books the encoder swap is the single biggest non-loss-component ablation, and is roughly the same magnitude as the projection-head ablation. Yelp and ML-1M either lack the structured text richness required to benefit from a bigger encoder in the first place, or are already saturating MiniLM-L6-v2’s 384-d capacity with the relatively small volume of structured metadata they have.

Table 2: Ablation grid for RLMRec+SASRec, 3 seeds $\times$ 50 epochs, NDCG@10 mean $\pm$ std. Significance vs. in-ablation reference `sasrec_full`: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$ (paired two-sided t -test). $\Delta\%$ is the relative change in NDCG@10.

Variant	MovieLens-1M	Yelp	Amazon-Books
<i>Loss components</i>			
RLMRec+SASRec (full, ref.)	0.1042 $\pm$ 0.0019	0.0162 $\pm$ 0.0002	0.0348 $\pm$ 0.0007
no user-align ($\lambda_u = 0$)	0.0925 $\pm$ 0.0022 (−11.2%)*	0.0165 $\pm$ 0.0004 (+2.1%)	0.0237 $\pm$ 0.0002 (−32.0%)**
no item-align ($\lambda_i = 0$)	0.1088 $\pm$ 0.0038 (+4.5%)	0.0137 $\pm$ 0.0005 (−15.7%)*	0.0318 $\pm$ 0.0005 (−8.7%)*
no align ($\sim$ pure SASRec)	0.1084 $\pm$ 0.0026 (+4.1%)	0.0121 $\pm$ 0.0004 (−25.3%)**	0.0213 $\pm$ 0.0010 (−38.8%)**
<i>Alignment weight λ sweep</i>			
$\lambda = 0.01$	0.1105 $\pm$ 0.0017 (+6.1%)	0.0148 $\pm$ 0.0001 (−8.9%)**	0.0280 $\pm$ 0.0005 (−19.5%)**
$\lambda = 0.05$	0.1108 $\pm$ 0.0015 (+6.4%)*	0.0165 $\pm$ 0.0002 (+1.7%)	0.0343 $\pm$ 0.0002 (−1.5%)
$\lambda = 0.5$	0.0746 $\pm$ 0.0011 (−28.4%)**	0.0136 $\pm$ 0.0002 (−16.0%)**	0.0310 $\pm$ 0.0001 (−11.0%)*
$\lambda = 1.0$	0.0639 $\pm$ 0.0013 (−38.7%)**	0.0120 $\pm$ 0.0001 (−26.0%)**	0.0282 $\pm$ 0.0003 (−19.0%)**
<i>InfoNCE temperature τ sweep</i>			
$\tau = 0.05$	0.1012 $\pm$ 0.0012 (−2.9%)	0.0151 $\pm$ 0.0007 (−7.0%)	0.0325 $\pm$ 0.0005 (−6.6%)**
$\tau = 0.2$	0.1077 $\pm$ 0.0022 (+3.4%)	0.0167 $\pm$ 0.0004 (+3.1%)	0.0346 $\pm$ 0.0007 (−0.6%)*
$\tau = 0.5$	0.1107 $\pm$ 0.0032 (+6.3%)	0.0156 $\pm$ 0.0003 (−3.8%)	0.0315 $\pm$ 0.0007 (−9.5%)*
<i>Architecture / encoder</i>			
linear proj-head	0.1015 $\pm$ 0.0035 (−2.6%)	0.0160 $\pm$ 0.0002 (−1.2%)	0.0283 $\pm$ 0.0003 (−18.6%)**
MiniLM-L6-v2 encoder	0.0970 $\pm$ 0.0027 (−6.9%)	0.0157 $\pm$ 0.0002 (−3.3%)	0.0278 $\pm$ 0.0004 (−20.1%)**

5.3 Comparison to original papers

SASRec paper baseline. The original SASRec paper reports HR@10 0.7022 and NDCG@10 0.4534 on ML-1M, but these numbers are computed under a sampled-100-negative evaluation protocol, not full-rank evaluation. Under full-rank LOO, my SASRec reaches HR@10 0.2143 and NDCG@10 0.1126, which falls well within the typical reproduction range of 0.18–0.22 HR@10 and 0.10–0.13 NDCG@10 reported by RecBole [Zhao et al., 2021] and by the analysis of Petrov and Macdonald [2023] for full-rank evaluation. The two protocols are simply not directly comparable, and the apparent “drop” from 0.70 to 0.21 is an artefact of the protocol change, not a regression in my implementation.

RLMRec paper baseline on Amazon-Books / Yelp. The RLMRec paper (LightGCN backbone) reports NDCG@20 0.0540 on Yelp and 0.0612 on Amazon-Books for the bare LightGCN baseline, lifting to 0.0594 (+10%) on Yelp and 0.0670 (+9%) on Amazon-Books after the contrastive alignment is added. My paper-faithful RLMRec+LightGCN reaches NDCG@20 0.0202 on Yelp and 0.0166 on Amazon-Books. The absolute numbers are lower in my setup for two reasons: (i) our preprocessing keeps a wider time window (start year 2018 rather than the paper’s narrower window) and uses a lighter k-core filter (5 rather than the paper’s heavier filter, which inflates per-user density), and (ii) my templated profile text is a weaker signal than the LLM-authored profile in the paper. The *relative* lift of RLMRec+SASRec over SASRec (+30.8% on Yelp NDCG@10, +60.0% on Amazon-Books) is in fact substantially larger than the paper’s LightGCN lift, which suggests that the contrastive alignment has more to contribute when the backbone is sequential — the alignment fills in a kind of structural information that LightGCN’s graph propagation already partially captures, but that a sequential transformer genuinely cannot reconstruct from interaction order alone.

6 Conclusion

6.1 Summary of findings

We have shown that the contrastive cross-view alignment introduced by RLMRec [Ren et al., 2024] for graph-CF backbones transfers cleanly to a self-attentive sequential backbone [Kang and McAuley, 2018], *provided* the underlying dataset is in the regime RLMRec was originally designed for: sparse interactions, rich item-side text. On Amazon-Books, adding two thin MLP projection heads and an InfoNCE objective on top of SASRec’s last-position hidden state and item embedding table lifts NDCG@10 by +60.0% and HR@10 by +50.6% relative to a vanilla SASRec backbone with the same training configuration. On Yelp, the same recipe yields +30.8% NDCG@10 and +28.7% HR@10. On the dense, short-text MovieLens-1M dataset, the alignment loss is approximately neutral, with a small −3.3% NDCG@10 regression that I attribute to the very limited semantic

content of ML-1M’s profile text. My 14-variant ablation grid further shows that (i) the user- and item-alignment heads contribute *asymmetrically*, with the asymmetry tracking the relative richness of user- vs. item-side profile text on each dataset; (ii) the InfoNCE temperature is a relatively robust knob, with the τ -sweep staying within $\pm 10\%$ on every dataset; and (iii) the non-linear projection head and the choice of sentence encoder (BGE-large-en-v1.5 over MiniLM-L6-v2) account for a non-trivial portion of the headline gain on text-rich datasets.

6.2 Implications

The architectural recipe is a minimal change to any existing SASRec deployment. The engineering footprint is essentially: load a frozen sentence encoder, encode the already-existing user and item metadata once, cache the resulting embeddings on disk, and add two trainable MLPs plus a single line of InfoNCE loss to the training step. This is a much smaller commitment than joint LLM fine-tuning or full LLM-as-recommender pipelines, and my results suggest that a sizable fraction of the benefit of richer semantic priors is recoverable at this cost.

A second implication is methodological: the original RLMRec contribution is largely backbone-agnostic, but it is also dataset-agnostic only *up to the richness of the profile text*. The variant-vs-reference Δ column in Table 2 can therefore be used as a quick diagnostic for whether a new dataset has enough item text to benefit from contrastive alignment before committing to a full ablation grid. If removing the user- and item-alignment heads is statistically insignificant on a small pilot, the default RLMRec+SASRec configuration is unlikely to help on the full dataset either.

6.3 Limitations

Three limitations qualify my results. First, my profile text is template-based, not LLM-authored; the gains RLMRec originally reported depend on a higher-quality LLM-generated profile that we chose not to pay for. The absolute gains I report on Yelp and Amazon-Books are therefore likely a *lower bound* on what the recipe can deliver. Second, I did not tune the SASRec backbone hyperparameters per dataset — the literature on Amazon-Books in particular suggests that a slightly higher dropout or a longer maximum sequence length helps. I left these defaults alone deliberately, to isolate the effect of the alignment loss and avoid confounding the comparison. Third, my evaluation is single-step next-item recommendation, which is the protocol both SASRec and RLMRec target, but downstream multi-step retrieval or session-level metrics may behave differently.

6.4 Future work

Several natural extensions follow from the present work. **First**, the most direct experiment is to replace the templated profile builder with an LLM-authored one — using e.g. Claude or GPT-4 to write user / item profiles — and re-measure. I expect the Yelp and Amazon-Books lifts to grow further, and I expect the ML-1M result to plausibly flip positive once each movie’s profile is a paragraph instead of three words. **Second**, it would be interesting to test whether *generative* alignment (RLMRec-Gen, the masked-autoencoder variant in the original paper) further improves the sequential case — my contrastive head is strictly the simpler of the two objectives the original RLMRec paper studied. **Third**, a multi-encoder consensus setup is left on the table: one could have multiple frozen sentence encoders (a small one and a large one, or one English and one multilingual) each contribute a separate alignment head, trading off encoder dimension against representational diversity. **Fourth**, the auxiliary heads suggest a natural cold-start story: at inference time, an item unseen during training has no learned ID embedding, but it does have a semantic embedding, and the item-projection head could in principle be inverted to bootstrap a synthetic CF embedding for the new item. I did not benchmark cold-start in this paper because all three datasets share a common 5-core filter that excludes the tail, but it is a clean direction for follow-up work.

Reproducibility statement

The full code, datasets, semantic-embedding cache layout, and run configurations used in this paper are well documented at <https://github.com/DavWexler/RLMRec-SASRec>. A single python `run_everything.py` command reproduces Table 1, and python `run_ablations.py` reproduces Table 2. Seeds are fixed for both NumPy and PyTorch; the dataset preprocessing is deterministic

given the `start_year / min_inter` flags. The `run_metadata.json` file in each per-seed artifact directory records the full configuration used.

References

- Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, pages 1007–1014, 2023.
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020.
- F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015. doi: 10.1145/2827872.
- Jesse Harte, Wouter Zorgdrager, Panos Louridas, Asterios Katsifodimos, Dietmar Jannach, and Marios Fragkoulis. Leveraging large language models for sequential recommendation. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, pages 1096–1102, 2023.
- Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the International Conference on World Wide Web (WWW)*, pages 173–182, 2017. doi: 10.1145/3038912.3052569.
- Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 639–648, 2020. doi: 10.1145/3397271.3401063.
- Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2016.
- Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. Bridging language and items for retrieval and recommendation. In *arXiv preprint arXiv:2403.03952*, 2024a.
- Yupeng Hou, Zhicheng Liao, Yixin Li, Wayne Xin Zhao, and Julian McAuley. LLaRA: Large language-recommendation assistant. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024b.
- Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. Large language models are zero-shot rankers for recommender systems. In *Proceedings of the European Conference on Information Retrieval (ECIR)*, 2024c.
- Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pages 197–206. IEEE, 2018. doi: 10.1109/ICDM.2018.00035.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Xiangyang Li, Bo Chen, Lu Hou, and Ruiming Tang. CTRL: Connect tabular and language model for CTR prediction. In *arXiv preprint arXiv:2306.02841*, 2023.
- Aleksandr V. Petrov and Craig Macdonald. A systematic review and replicability study of BERT4Rec for sequential recommendation. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, 2023. doi: 10.1145/3604915.3609717.
- Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. Representation learning with large language models for recommendation. In *Proceedings of the ACM Web Conference (WWW)*, 2024. doi: 10.1145/3589334.3645458.

- Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 452–461, 2009.
- Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. Factorizing personalized Markov chains for next-basket recommendation. In *Proceedings of the International Conference on World Wide Web (WWW)*, pages 811–820, 2010. doi: 10.1145/1772690.1772773.
- Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1441–1450, 2019. doi: 10.1145/3357384.3357895.
- Jiaxi Tang and Ke Wang. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM)*, pages 565–573, 2018. doi: 10.1145/3159652.3159656.
- Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.
- Lei Wang and Ee-Peng Lim. Zero-shot next-item recommendation using large pretrained language models. In *arXiv preprint arXiv:2304.03153*, 2023.
- Yunjia Xi, Weiwen Liu, Jianghao Lin, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, Rui Zhang, and Yong Yu. Towards open-world recommendation with knowledge augmentation from large language models. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, 2024.
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.
- Yelp Inc. Yelp Open Dataset. <https://www.yelp.com/dataset>, 2022. Accessed 2025-03.
- Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. RecBole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, pages 4653–4664, 2021. doi: 10.1145/3459637.3482016.